



Documentación Completa del Proyecto

Índice

1. [Descripción General](#)
 2. [Stack Tecnológico](#)
 3. [Estructura del Proyecto](#)
 4. [Configuración Detallada](#)
 5. [Guía de Uso](#)
 6. [Flujo de Trabajo Completo](#)
 7. [Comandos Disponibles](#)
 8. [Convenciones y Mejores Prácticas](#)
 9. [Troubleshooting](#)
 10. [Recursos Adicionales](#)
-

Descripción General

Este es un **starter project moderno de React + TypeScript + Vite** configurado con las mejores prácticas y herramientas más actuales del ecosistema JavaScript/TypeScript. El proyecto está optimizado para desarrollo profesional con un enfoque en:

- **Rendimiento:** Build ultra-rápido con Vite y React Compiler
- **Calidad de Código:** TypeScript estricto, ESLint, Prettier
- **Testing:** Vitest para unit testing y Playwright para E2E
- **Git Workflow:** Lefthook, Commitlint, Commitizen, Lint-staged
- **Desarrollador Experience:** Path aliases, HMR, intellisense completo

Objetivo del Proyecto

Proporcionar una base sólida y moderna para desarrollar aplicaciones React escalables, mantenibles y con alta calidad de código, permitiendo a los desarrolladores enfocarse en la lógica de negocio en lugar de la configuración.

⚡ Stack Tecnológico

Core Technologies

Tecnología	Versión	Propósito
React	19.2.6	Biblioteca UI para construir interfaces de usuario
TypeScript	6.0.2	Superset de JavaScript con tipado estático
Vite	8.0.12	Build tool y dev server ultrarrápido

Development Tools

Herramienta	Versión	Propósito
ESLint	10.4.1	Linting de código JavaScript/TypeScript
Prettier	3.8.3	Formateo de código consistente
TypeScript ESLint	8.59.2	Reglas ESLint específicas para TypeScript

Testing Frameworks

Herramienta	Versión	Propósito
Vitest	4.1.8	Framework de testing unitario
Playwright	1.60.0	Framework de testing E2E

Herramienta	Versión	Propósito
Testing Library	16.3.2	Utilidades para testing de componentes React

Git & Quality Tools

Herramienta	Versión	Propósito
Lefthook	2.1.9	Git hooks manager
Commitlint	21.0.2	Linting de mensajes de commit
Commitizen	4.3.1	Interactivo para commits convencionales
Lint-staged	17.0.7	Ejecuta linters solo en archivos staged

Performance Tools

Herramienta	Versión	Propósito
React Compiler	1.0.0	Compilador automático de optimización de React
👉 @rolldown/plugin-babel	0.2.3	Plugin Babel para Vite

Estructura del Proyecto

```
react-starter-pnpm/  
├─ .devin/                # Configuración de Devin CLI  
├─ .git/                  # Repositorio Git  
├─ .idea/                 # Configuración de IntelliJ/  
WebStorm  
├─ .qodo/                 # Configuración de Qodo AI  
├─ node_modules/          # Dependencias instaladas  
├─ src/                   # Código fuente de la aplicación  
│   └─ App.tsx            # Componente principal de la  
aplicación  
│   └─ App.test.tsx       # Tests del componente App  
│   └─ main.tsx           # Punto de entrada de la aplicación  
│   └─ setupTests.ts      # Configuración global de tests  
└─ dist/                  # Build de producción (generado)
```

— .czrc	# Configuración de Commitizen
— .gitignore	# Archivos ignorados por Git
— .lintstagedrc.json	# Configuración de Lint-staged
— .prettierrc.json	# Configuración de Prettier
— .prettierignore	# Archivos ignorados por Prettier
— commitlint.config.js	# Configuración de Commitlint
— docker-compose.yml	# Configuración Docker Compose
— Dockerfile	# Configuración Docker
— DOCUMENTATION.md	# Esta documentación
— eslint.config.js	# Configuración de ESLint
— index.html	# HTML entry point
— lefthook.yml	# Configuración de Lefthook
— nginx.conf	# Configuración Nginx
— package.json	# Dependencias y scripts
— pnpm-lock.yaml	# Lockfile de dependencias
— README.md	# Documentación básica
— tsconfig.json	# Configuración principal de TypeScript
— vite.config.ts	# Configuración de Vite
— vitest.config.ts	# Configuración de Vitest

Descripción de Directorios Principales

/src - Código Fuente

Contiene todo el código fuente de la aplicación React: - **Componentes:** Componentes React (.tsx) - **Hooks:** Custom hooks de React - **Utils:** Funciones utilitarias - **Types:** Definiciones de tipos TypeScript - **Assets:** Imágenes, fuentes, estilos - **Tests:** Archivos de testing

Archivos de Configuración Principales

- **package.json:** Define dependencias, scripts y metadatos del proyecto
 - **tsconfig.json:** Configuración del compilador TypeScript
 - **vite.config.ts:** Configuración del build tool Vite
 - **eslint.config.js:** Reglas de linting de código
 - **vitest.config.ts:** Configuración del framework de testing
-

Configuración Detallada

1. TypeScript Configuration

tsconfig.json - Configuración Principal

```
{
  "compilerOptions": {
    // Target y Module
    "target": "ES2023", // Versión de ECMAScript
    objetivo
    "lib": ["ES2023", "DOM", "DOM.Iterable"], // Librerías
    disponibles
    "module": "ESNext", // Sistema de módulos

    // Resolution
    "moduleResolution": "bundler", // Resolución de módulos
    para bundlers
    "resolveJsonModule": true, // Permite importar JSON
    "allowImportingTsExtensions": true, // Permite imports con
    extensión .ts

    // Output
    "noEmit": true, // No genera archivos de
    salida
    "isolatedModules": true, // Cada archivo es un módulo
    aislado
    "jsx": "react-jsx", // Transformación JSX
    automática

    // Type Checking - Modo Estricto
    "strict": true, // Habilita todas las
    verificaciones estrictas
    "noUnusedLocals": true, // Error en variables no
    usadas
    "noUnusedParameters": true, // Error en parámetros no
    usados
    "noImplicitReturns": true, // Error en retornos
    implícitos
    "noImplicitOverride": true, // Requiere override
    explícito
    "noUncheckedIndexedAccess": true, // Verificación de acceso a
    índices
    "noFallthroughCasesInSwitch": true, // Error en fallthrough
    de switch
    "noUncheckedSideEffectImports": true, // Verifica imports
    con side effects

    // Quality
```

```

"skipLibCheck": true,           // Omite verificación de
                                librerías
"allowUnusedLabels": false,     // Prohíbe labels no usados
"allowUnreachableCode": false,  // Prohíbe código
                                inalcanzable

// Modern TypeScript
"ignoreDeprecations": "6.0",    // Ignora advertencias de
                                deprecación

// Path Aliases
"baseUrl": ".",                 // Directorio base para
                                paths relativos
"paths": {
  "@/*": ["/src/*"],            // Alias para directorio src
  "@components/*": ["/src/components/*"],
  "@hooks/*": ["/src/hooks/*"],
  "@utils/*": ["/src/utils/*"],
  "@types/*": ["/src/types/*"],
  "@assets/*": ["/src/assets/*"],
  "@tests/*": ["/tests/*"]
},
"include": ["src", "vitest.config.ts", "vite.config.ts"],
"exclude": ["node_modules", "dist", "**/*.test.ts", "**/
            *.test.tsx"]
}

```

¿Para qué sirven estas opciones?

Opciones de Target y Module: - target: "ES2023": Compila a JavaScript moderno ES2023 - module: "ESNext": Usa el sistema de módulos más reciente - lib: Define qué APIs globales están disponibles

Opciones de Resolución: - moduleResolution: "bundler": Optimizado para herramientas como Vite - allowImportingTsExtensions: Permite import { x } from './file.ts' sin renombrar

Opciones de Type Checking (Modo Estricto): - strict: true: Activa todas las verificaciones de tipo estrictas - noUnusedLocals/Parameters: Evita código muerto - noUncheckedIndexedAccess: Previene errores de undefined en arrays - noImplicitReturns: Asegura que todas las funciones retornen algo

Path Aliases:

```
// Sin alias
import { Button } from '../../../components/Button'

// Con alias
import { Button } from '@components/Button'
```

2. Vite Configuration

vite.config.ts - Configuración del Build Tool

```
import { defineConfig } from 'vite'
import react, { reactCompilerPreset } from '@vitejs/plugin-react'
import babel from '@rollup/plugin-babel'
import { resolve } from 'path'

export default defineConfig({
  plugins: [
    react(), // Plugin oficial de React para Vite
    babel({
      presets: [reactCompilerPreset()] // React Compiler
    })
  ],
  resolve: {
    alias: {
      // Path aliases (deben coincidir con tsconfig.json)
      '@': resolve(__dirname, './src'),
      '@components': resolve(__dirname, './src/components'),
      '@hooks': resolve(__dirname, './src/hooks'),
      '@utils': resolve(__dirname, './src/utils'),
      '@types': resolve(__dirname, './src/types'),
      '@assets': resolve(__dirname, './src/assets'),
      '@tests': resolve(__dirname, './tests'),
    },
  },
})
```

Plugins de Vite

@vitejs/plugin-react: - Proporciona soporte para React en Vite -
Incluye HMR (Hot Module Replacement) para React - Optimiza imports de React - Soporta JSX y TSX

@rolldown/plugin-babel con React Compiler: - React Compiler optimiza automáticamente componentes React - Elimina la necesidad de useMemo, useCallback en muchos casos - Mejora el rendimiento mediante análisis estático

¿Qué hace Vite?

1. **Dev Server:** Servidor de desarrollo con HMR ultra-rápido
2. **Build Optimizado:** Genera bundles optimizados para producción
3. **Transformación:** Convierte TypeScript/JSX a JavaScript
4. **Code Splitting:** Divide el código automáticamente
5. **Asset Handling:** Procesa imágenes, fuentes, CSS automáticamente

3. ESLint Configuration

eslint.config.js - Configuración de Linting

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import tseslint from 'typescript-eslint'
import react from 'eslint-plugin-react'
import jsxA11y from 'eslint-plugin-jsx-a11y'
import eslintConfigPrettier from 'eslint-config-prettier'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist', 'node_modules', 'build', 'coverage']),
  {
    files: ['**/*.ts', '**/*.tsx'],
    extends: [
      js.configs.recommended, // Reglas
      js.configs.basics, // Reglas de JS
      tseslint.configs.recommended, // Reglas de
      tseslint.configs.strict, // Reglas de TypeScript
      reactHooks.configs.flat.recommended, // Reglas de
      reactHooks.configs.flat.strict, // Reglas de React Hooks
      reactRefresh.configs.vite, // Optimización
      reactRefresh.configs.hmr, // Optimización HMR
      react.configs.flat.recommended, // Reglas de
      react.configs.flat.strict, // Reglas de React
      jsxA11y.configs.flat.recommended, // Reglas de
      jsxA11y.configs.flat.strict, // Reglas de accesibilidad
    ],
  },
])
```



```

],
languageOptions: {
  globals: globals.browser,           // Variables
  globales del browser
  parserOptions: {
    ecmaVersion: 'latest',
    sourceType: 'module',
    ecmaFeatures: {
      jsx: true,                       // Soporte JSX
    },
  },
},
settings: {
  react: {
    version: '19.2.6',                // Versión de
    React
  },
},
rules: {
  // React Refresh
  'react-refresh/only-export-components': [
    'warn',
    { allowConstantExport: true },
  ],

  // React
  'react/prop-types': 'off',           // No requerir
  PropTypes
  'react/react-in-jsx-scope': 'off',   // React 19 no
  requiere import
  'react/jsx-uses-react': 'off',        // React 19 no
  requiere import
  'react/jsx-no-target-blank': 'error', // Seguridad en
  links
  'react/no-array-index-key':
    'warn', // Evitar keys de índice
  'react/jsx-key': 'error',             // Requerir keys
  en listas
  'react/no-unescaped-entities':
    'warn', // Entidades HTML escapadas

  // React Hooks
  'react-hooks/rules-of-hooks': 'error', // Reglas de
  Hooks
  'react-hooks/exhaustive-deps': 'warn', // Dependencias
  de useEffect

  // TypeScript

```

```

    '@typescript-eslint/no-unused-vars': [
      'error',
      {
        argsIgnorePattern: '^_', // Ignorar args
        que empiezan con _
        varsIgnorePattern: '^_', // Ignorar vars
        que empiezan con _
        caughtErrorsIgnorePattern: '^_', // Ignorar
        errores catch
      },
    ],
    '@typescript-eslint/no-explicit-any': 'warn', // Advertir
    sobre any
    '@typescript-eslint/explicit-function-return-type':
      'off', // No requerir tipos de retorno
    '@typescript-eslint/explicit-module-boundary-types':
      'off', // No requerir tipos en exports
    '@typescript-eslint/no-non-null-assertion': 'warn', //
    Advertir sobre !
  },
},
eslintConfigPrettier, // Desactiva reglas que conflictúan con
Prettier
])

```

Reglas Importantes Explicadas

Reglas de React: - react/jsx-key: Requiere keys en listas para evitar errores de rendering - react/jsx-no-target-blank: Previene vulnerabilidades de seguridad en links externos - react/no-array-index-key: Advierte sobre usar índices como keys (anti-pattern)

Reglas de React Hooks: - react-hooks/rules-of-hooks: Asegura que los Hooks se usen correctamente - react-hooks/exhaustive-deps: Verifica dependencias en useEffect, useCallback, etc.

Reglas de TypeScript: - @typescript-eslint/no-unused-vars: Detecta variables no usadas - @typescript-eslint/no-explicit-any: Advierte sobre el uso de any

4. Prettier Configuration

.prettierrc.json - Configuración de Formateo

```

{
  "trailingComma": "all", // Comas al final en objetos/
  arrays
  "tabWidth": 2, // Tamaño de indentación
}

```

```

"printWidth": 80, // Ancho máximo de línea
"semi": true, // Punto y coma al final
"jsxSingleQuote": false, // Comillas dobles en JSX
"singleQuote": true, // Comillas simples en strings
"useTabs": false, // Usar espacios en lugar de
    tabs
"endOfLine": "lf", // Saltos de línea LF (Unix)
"bracketSpacing": true, // Espacios en objetos: { a:
    1 }
"arrowParens": "always", // Paréntesis en arrow
    functions: (x) => x
"bracketSameLine": false, // } en nueva línea
"proseWrap": "preserve", // Preservar saltos en
    markdown
"quoteProps": "as-needed", // Comillas en props solo
    cuando es necesario
"htmlWhitespaceSensitivity": "css", // Sensibilidad a
    whitespace en HTML
"embeddedLanguageFormatting": "auto" // Formatear código
    embebido
}

```

¿Qué hace Prettier?

- **Formateo automático:** Mantiene el estilo consistente
- **Sin configuración de estilo:** No requiere discusiones sobre estilo
- **Integración:** Funciona con ESLint, editores, pre-commit hooks

5. Vitest Configuration

vitest.config.ts - Configuración de Testing

```

import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'
import { resolve } from 'path'

export default defineConfig({
  plugins: [react()],
  test: {
    globals: true, // APIs globales (describe,
    it, expect)
    environment: 'jsdom', // Environment de navegador
    simulado
    setupFiles: ['./src/setupTests.ts'], // Archivo de
    configuración
    css: true, // Soporte para CSS
    coverage: {

```

```

    provider: 'v8', // Proveedor de coverage
    reporter: ['text', 'json', 'html'], // Formatos de reporte
    exclude: [
      'node_modules/',
      'src/setupTests.ts',
      '**/*.d.ts',
      '**/*.config.*',
      '**/mockData',
      'dist/',
      'build/',
      'coverage/',
    ],
  },
},
resolve: {
  alias: {
    // Path aliases (deben coincidir con vite.config.ts)
    '@': resolve(__dirname, './src'),
    '@components': resolve(__dirname, './src/components'),
    '@hooks': resolve(__dirname, './src/hooks'),
    '@utils': resolve(__dirname, './src/utils'),
    '@types': resolve(__dirname, './src/types'),
    '@assets': resolve(__dirname, './src/assets'),
    '@tests': resolve(__dirname, './tests'),
  },
},
})

```

src/setupTests.ts - Configuración Global de Tests

```
import '@testing-library/jest-dom'
```

Este archivo importa @testing-library/jest-dom que proporciona matchers personalizados como: - toBeInTheDocument() - toHaveTextContent() - toBeDisabled() - etc.

6. Git Hooks Configuration

lefthook.yml - Git Hooks Manager

```

pre-commit:
  parallel: true # Ejecutar comandos en paralelo
  commands:
    lint-staged:

```

```
run: npx lint-staged # Ejecutar linters en archivos staged

commit-msg:
  commands:
    commitlint:
      run: npx commitlint --edit {1} # Validar mensaje de commit
```

¿Qué hacen estos hooks?

Pre-commit: - Se ejecuta antes de crear un commit - Ejecuta lint-staged que aplica ESLint y Prettier a los archivos modificados - Si hay errores, previene el commit

Commit-msg: - Valida el formato del mensaje de commit - Asegura que sigue el formato convencional - Si el formato es incorrecto, previene el commit

.lintstagedrc.json - Configuración de Lint-staged

```
{
  "{*.ts,tsx,js,jsx}": [
    "eslint --fix",      # Corregir errores de ESLint
                        automáticamente
    "prettier --write"   # Formatear con Prettier
  ],
  "{*.json,md,yml,yaml}": [
    "prettier --write"   # Formatear archivos de configuración
  ]
}
```

commitlint.config.js - Validación de Commits

```
export default {
  extends: ['@commitlint/config-conventional'],
}
```

Formato Convencional de Commits

<tipo>(<alcance>): <descripción>

[opcional cuerpo]

[opcional footer]

Tipos permitidos: - feat: Nueva funcionalidad - fix: Corrección de bug
- docs: Cambios en documentación - style: Cambios de estilo
(formatting, whitespace) - refactor: Refactorización de código - perf:
Mejoras de performance - test: Agregar o actualizar tests - chore:
Cambios en build process o herramientas

Ejemplos:

```
feat(auth): add login form  
fix(button): resolve click handler issue  
docs(readme): update installation guide
```

.czrc - Configuración de Commitizen

```
{  
  "path": "cz-conventional-changelog"  
}
```

Commitizen proporciona una interfaz interactiva para crear commits que siguen el formato convencional.

7. Playwright Configuration (E2E Testing)

Playwright está configurado pero requiere inicialización. Para configurarlo:

```
pnpm exec playwright install
```

Playwright permite: - Testing E2E en múltiples navegadores (Chrome, Firefox, Safari) - Testing de APIs - Generación automática de tests - Visual regression testing

Guía de Uso

Instalación y Setup Inicial

1. Instalar Dependencias

```
pnpm install
```

Este comando: - Lee package.json - Instala todas las dependencias listadas - Crea/actualiza pnpm-lock.yaml - Instala dependencias en node_modules/

2. Iniciar Servidor de Desarrollo

pnpm dev

Esto iniciará: - Servidor de desarrollo Vite en <http://localhost:5173> - Hot Module Replacement (HMR) activo - Compilación de TypeScript al vuelo - Source maps para debugging

3. Abrir en el Navegador

Visita <http://localhost:5173> para ver tu aplicación.

Desarrollo Diario

Crear un Nuevo Componente

```
// src/components/Button.tsx
interface ButtonProps {
  children: React.ReactNode
  onClick?: () => void
  variant?: 'primary' | 'secondary'
}

export function Button({ children, onClick, variant =
  'primary' }: ButtonProps) {
  return (
    <button
      onClick={onClick}
      className={`button button-${variant}`}
    >
      {children}
    </button>
  )
}
```

Crear Test para el Componente

```
// src/components/Button.test.tsx
import { describe, it, expect } from 'vitest'
import { render, screen, fireEvent } from '@testing-library/react'
import { Button } from './Button'

describe('Button', () => {
  it('renders children correctly', () => {
```

```

    render(<Button>Click me</Button>)
    expect(screen.getByText('Click me')).toBeInTheDocument()
  })

  it('calls onClick when clicked', () => {
    const handleClick = vi.fn()
    render(<Button onClick={handleClick}>Click me</Button>)

    fireEvent.click(screen.getByText('Click me'))
    expect(handleClick).toHaveBeenCalledTimes(1)
  })
})

```

Usar Path Aliases

```

// Sin alias (no recomendado)
import { Button } from '../../../components/Button'
import { useAuth } from '../../../hooks/useAuth'

// Con alias (recomendado)
import { Button } from '@components/Button'
import { useAuth } from '@hooks/useAuth'

```

Crear un Custom Hook

```

// src/hooks/useCounter.ts
import { useState, useCallback } from 'react'

export function useCounter(initialValue: number = 0) {
  const [count, setCount] = useState(initialValue)

  const increment = useCallback(() => {
    setCount(prev => prev + 1)
  }, [])

  const decrement = useCallback(() => {
    setCount(prev => prev - 1)
  }, [])

  const reset = useCallback(() => {
    setCount(initialValue)
  }, [initialValue])
}

```



```
    return { count, increment, decrement, reset }  
  }  
}
```

Crear Utilidades

```
// src/utils/formatDate.ts  
export function formatDate(date: Date): string {  
  return new Intl.DateTimeFormat('es-ES', {  
    year: 'numeric',  
    month: 'long',  
    day: 'numeric'  
  }).format(date)  
}  
  
export function formatCurrency(amount: number): string {  
  return new Intl.NumberFormat('es-ES', {  
    style: 'currency',  
    currency: 'EUR'  
  }).format(amount)  
}
```

Crear Tipos

```
// src/types/user.ts  
export interface User {  
  id: string  
  name: string  
  email: string  
  role: 'admin' | 'user' | 'guest'  
  createdAt: Date  
}  
  
export interface UserResponse {  
  data: User[]  
  total: number  
  page: number  
}  
  
export type UserRole = User['role']
```

Testing

Ejecutar Tests en Modo Watch

```
pnpm test
```

Esto iniciará Vitest en modo watch: - Ejecuta tests automáticamente cuando cambian los archivos - Muestra resultados en tiempo real - Permite filtrar tests por patrón

Ejecutar Tests una vez

```
pnpm test --run
```

Ejecutar Tests con UI

```
pnpm test:ui
```

Abre una interfaz visual para: - Ver resultados de tests - Filtrar y ejecutar tests específicos - Ver coverage de código

Ejecutar Coverage

```
pnpm coverage
```

Genera un reporte de coverage: - Líneas de código cubiertas - Branches cubiertas - Funciones cubiertas - Statements cubiertos

```
pnpm coverage:report
```

Genera reportes en múltiples formatos (text, json, html).

```
pnpm coverage:open
```

Abre el reporte HTML en el navegador.

Ejemplo de Test Complejo

```
// src/components/UserList.test.tsx
import { describe, it, expect, beforeEach } from 'vitest'
import { render, screen, waitFor } from '@testing-library/react'
import { UserList } from '../UserList'
import { fetchUsers } from '@utils/api'

// Mock de la API
vi.mock('@utils/api', () => ({
```

```

    fetchUsers: vi.fn(),
  }))

describe('UserList', () => {
  const mockUsers = [
    { id: '1', name: 'John', email: 'john@example.com' },
    { id: '2', name: 'Jane', email: 'jane@example.com' },
  ]

  beforeEach(() => {
    vi.clearAllMocks()
  })

  it('shows loading state initially', () => {
    vi.mocked(fetchUsers).mockImplementation(() => new Promise(() => {}))

    render(<UserList />)
    expect(screen.getByText('Loading...')).toBeInTheDocument()
  })

  it('displays users after loading', async () => {
    vi.mocked(fetchUsers).mockResolvedValue(mockUsers)

    render(<UserList />)

    await waitFor(() => {
      expect(screen.getByText('John')).toBeInTheDocument()
      expect(screen.getByText('Jane')).toBeInTheDocument()
    })
  })

  it('shows error message on failure', async () => {
    vi.mocked(fetchUsers).mockRejectedValue(new Error('API Error'))

    render(<UserList />)

    await waitFor(() => {
      expect(screen.getByText('Error loading users')).toBeInTheDocument()
    })
  })
})

```

E2E Testing con Playwright

Inicializar Playwright

```
pnpm exec playwright install
```

Crear Test E2E

```
// tests/e2e/example.spec.ts
import { test, expect } from '@playwright/test'

test('homepage loads correctly', async ({ page }) => {
  await page.goto('http://localhost:5173')

  await expect(page).toHaveTitle(/React Starter/)
  await expect(page.locator('h1')).toContainText('React Starter')
})

test('navigation works', async ({ page }) => {
  await page.goto('http://localhost:5173')

  await page.click('text=About')
  await expect(page).toHaveURL(/.*about/)
})
```

Ejecutar Tests E2E

```
pnpm test:e2e
```

Ejecutar Tests E2E con UI

```
pnpm test:e2e:ui
```

Git Workflow

Hacer Changes

```
# Editar archivos
git add .
```

Pre-commit Hook Automático

Cuando ejecutas `git commit`, los hooks se ejecutan automáticamente:

1. **Lint-staged** ejecuta ESLint y Prettier en los archivos modificados
2. Si hay errores de ESLint que no se pueden corregir automáticamente, el commit se rechaza
3. Si todo está correcto, el commit procede

Crear Commit con Commitizen

```
pnpm commit
```

Esto abrirá una interfaz interactiva:

```
? Select the type of change that you're committing:
feat:      A new feature
fix:       A bug fix
docs:      Documentation only changes
style:     Changes that do not affect the meaning of the code
refactor:  A code change that neither fixes a bug nor adds a
feature
perf:      A code change that improves performance
test:      Adding missing tests or correcting existing tests
chore:     Changes to the build process or auxiliary tools
```

Luego te pedirá: - Scope (alcance del cambio) - Description (descripción breve) - Body (descripción detallada, opcional) - Footer (issues relacionados, opcional)

Ejemplo de Commit Exitoso

```
$ pnpm commit
```

```
? Select the type of change that you're committing: feat
? What is the scope of this change? (e.g. component or file name):
  auth
? Write a short, imperative tense description of the change: add
  login form
? Provide a longer description of the change: (press enter to
  skip)

? Are there any breaking changes? No
? Does this change affect any open issues? No
```

```
[main abc1234] feat(auth): add login form  
1 file changed, 25 insertions(+)
```

Commit con Mensaje Directo

```
git commit -m "feat(auth): add login form"
```

Commitlint validará que el formato sea correcto.

Linting y Formateo

Lintear Manualmente

```
pnpm lint
```

Ejecuta ESLint en todos los archivos y muestra errores y advertencias.

Lintear y Corregir Automáticamente

```
pnpm lint --fix
```

ESLint corregirá automáticamente los errores que pueda.

Formatear con Prettier

```
npx prettier --write .
```

Formatea todos los archivos con Prettier.

Verificar Formateo

```
npx prettier --check .
```

Verifica si los archivos están formateados correctamente (sin modificarlos).

Build para Producción

Crear Build de Producción

```
pnpm build
```

Este comando: 1. Ejecuta TypeScript para verificar tipos 2. Ejecuta Vite para crear el build optimizado 3. Genera archivos en dist/ 4. Optimiza y minifica el código 5. Genera source maps

Estructura del Build

```
dist/
├── index.html           # HTML entry point
└── assets/
    ├── index-[hash].js  # JavaScript bundle
    └── index-[hash].css  # CSS (si existe)
```

Previsualizar Build de Producción

```
pnpm preview
```



Inicia un servidor local para previsualizar el build de producción.

Flujo de Trabajo Completo

Escenario 1: Desarrollar Nueva Funcionalidad

1. Crear Rama de Feature

```
git checkout -b feat/user-authentication
```

2. Desarrollar Componentes

```
# Crear componente de autenticación
```

```
touch src/components/AuthForm.tsx
```

```
# Editar el archivo con tu código
```

3. Crear Tests

```
# Crear archivo de tests
```

```
touch src/components/AuthForm.test.tsx
```

```
# Escribir tests
```

4. Ejecutar Tests en Desarrollo

```
# Terminal 1: Servidor de desarrollo
pnpm dev
```

```
# Terminal 2: Tests en modo watch
pnpm test
```

5. Verificar Linting

```
pnpm lint
```

6. Hacer Commit

```
git add .
pnpm commit
```

Commitizen te guiará para crear un commit con formato correcto:

```
feat(auth): add user authentication form
```

7. Hacer Push y Crear Pull Request

```
git push origin feat/user-authentication
```

Luego crea un Pull Request en GitHub/GitLab.

Escenario 2: Corregir un Bug

1. Crear Rama de Fix

```
git checkout -b fix/login-button-error
```

2. Reproducir el Bug

```
pnpm dev
# Navegar a la página y reproducir el bug
```

3. Escribir Test que Falla

```
// src/components/LoginButton.test.tsx
it('should handle click correctly', () => {
  // Test que reproduce el bug
})
```



```
pnpm test # Debe fallar
```

4. Corregir el Bug

```
// src/components/LoginButton.tsx  
// Corregir el código
```

5. Verificar que el Test Pase

```
pnpm test # Debe pasar
```

6. Verificar Linting y Build

```
pnpm lint  
pnpm build
```

7. Commit con Tipo fix

```
git add .  
pnpm commit  
# Seleccionar tipo: fix  
# Scope: button  
# Descripción: resolve click handler issue
```

Escenario 3: Refactorización

1. Crear Rama de Refactor

```
git checkout -b refactor/optimize-user-service
```

2. Escribir Tests Primero (si no existen)

```
// src/services/user.service.test.ts  
describe('UserService', () => {  
  // Tests existentes para asegurar comportamiento  
})
```

3. Refactorizar Código

```
// src/services/user.service.ts  
// Mejorar estructura, performance, etc.
```

4. Verificar que Tests Pase

```
pnpm test
```

5. Verificar Linting

```
pnpm lint
```

6. Commit con Tipo refactor

```
git add .  
pnpm commit  
# Tipo: refactor  
# Scope: service  
# Descripción: optimize user service performance
```

Escenario 4: Actualizar Dependencias

1. Verificar Dependencias Desactualizadas

```
pnpm outdated
```

2. Actualizar Dependencia Específica

```
pnpm update react
```

3. Actualizar Todas las Dependencias

```
pnpm update
```

4. Verificar que Todo Funcione

```
pnpm lint  
pnpm test  
pnpm build
```

5. Commit con Tipo chore

```
git add .  
pnpm commit  
# Tipo: chore  
# Descripción: update dependencies
```

Comandos Disponibles

Desarrollo

Comando	Descripción
<code>pnpm dev</code>	Inicia servidor de desarrollo con HMR
<code>pnpm build</code>	Crea build de producción
<code>pnpm preview</code>	Previsualiza build de producción

Calidad de Código

Comando	Descripción
<code>pnpm lint</code>	Ejecuta ESLint para verificar código
<code>pnpm lint --fix</code>	Corrige automáticamente errores de ESLint

Testing

Comando	Descripción
<code>pnpm test</code>	Ejecuta tests en modo watch
<code>pnpm test --run</code>	Ejecuta tests una vez
<code>pnpm test:ui</code>	Ejecuta tests con interfaz visual
<code>pnpm test:e2e</code>	Ejecuta tests E2E con Playwright
<code>pnpm test:e2e:ui</code>	Ejecuta tests E2E con interfaz visual
<code>pnpm coverage</code>	Genera reporte de coverage
<code>pnpm coverage:report</code>	Genera reportes de coverage en múltiples formatos
<code>pnpm coverage:open</code>	Abre reporte de coverage HTML

Git

Comando	Descripción
<code>pnpm commit</code>	Crea commit con Commitizen (interactivo)

pnpm

Comando	Descripción
pnpm install	Instala dependencias
pnpm update	Actualiza dependencias
pnpm outdated	Muestra dependencias desactualizadas
pnpm add <package>	Agrega nueva dependencia
pnpm add -D <package>	Agrega nueva dependencia de desarrollo

Convenciones y Mejores Prácticas

Convenciones de Nomenclatura

Archivos

- **Componentes:** PascalCase (Button.tsx, UserProfile.tsx)
- **Hooks:** camelCase con prefijo use (useCounter.ts, useAuth.ts)
- **Utilidades:** camelCase (formatDate.ts, apiClient.ts)
- **Tipos:** camelCase (userTypes.ts, apiTypes.ts)
- **Tests:** Mismo nombre que el archivo con .test.tsx (Button.test.tsx)

Variables y Funciones

```
// Variables: camelCase
const userName = 'John'
const isLoggedIn = true

// Funciones: camelCase
function getUserData() { }
function handleSubmit() { }

// Constantes: UPPER_SNAKE_CASE
const API_BASE_URL = 'https://api.example.com'
const MAX_RETRIES = 3

// Interfaces y Types: PascalCase
interface User { }
type UserRole = 'admin' | 'user'
```

```
// Enumeraciones: PascalCase
enum UserRole {
  Admin = 'admin',
  User = 'user'
}
```

Estructura de Componentes

```
// 1. Imports
import React from 'react'
import { useAuth } from '@hooks/useAuth'
import { Button } from '@components/Button'
import type { UserProps } from '@types/user'

// 2. Types/Interfaces
interface MyComponentProps {
  title: string
  onClick: () => void
}

// 3. Componente
export function MyComponent({ title, onClick }: MyComponentProps)
{
  // 4. Hooks
  const { user } = useAuth()

  // 5. Handlers
  const handleClick = () => {
    onClick()
  }

  // 6. Effects
  React.useEffect(() => {
    // Lógica de efecto
  }, [])

  // 7. Render
  return (
    <div>
      <h1>{title}</h1>
      <Button onClick={handleClick}>Click me</Button>
    </div>
  )
}
```

Mejores Prácticas de React

1. Usar Componentes Funcionales

```
// Bueno
export function Button({ children }: { children:
  React.ReactNode }) {
  return <button>{children}</button>
}

// Evitar (class components)
export class Button extends React.Component {
  // ...
}
```

2. Usar TypeScript para Props

```
// Bueno
interface ButtonProps {
  children: React.ReactNode
  onClick?: () => void
  disabled?: boolean
}

export function Button({ children, onClick, disabled }:
  ButtonProps) {
  return (
    <button onClick={onClick} disabled={disabled}>
      {children}
    </button>
  )
}

// Evitar
export function Button(props: any) {
  // ...
}
```

3. Evitar any Type

```
// Bueno
interface User {
  id: string
  name: string
  email: string
}
```

```

}

x
function processUser(user: User) {
  // ...
}

// Evitar
function processUser(user: any) {
  // ...
}

```

4. Usar React Compiler

Con React Compiler habilitado, no necesitas:

- useMemo para memoización de valores
- useCallback para memoización de funciones
- React.memo para memoización de componentes

```

// Bueno (con React Compiler)
export function ExpensiveComponent({ data }: { data: Data[] }) {
  const processed = data.map(item => expensiveCalculation(item))
  return <div>{/* ... */</div>
}

// No necesario (con React Compiler)
export function ExpensiveComponent({ data }: { data: Data[] }) {
  const processed = useMemo(
    () => data.map(item => expensiveCalculation(item)),
    [data]
  )
  return <div>{/* ... */</div>
}

```

5. Manejo de Errores

```

// Bueno
export function UserProfile({ userId }: { userId: string }) {
  const { data, error, isLoading } = useUser(userId)

  if (isLoading) return <div>Loading...</div>
  if (error) return <div>Error: {error.message}</div>
  if (!data) return <div>User not found</div>

  return <div>{data.name}</div>
}

```

Mejores Prácticas de Testing

1. Tests Descriptivos

```
// Bueno
describe('Button', () => {
  it('renders children correctly', () => { })
  it('calls onClick when clicked', () => { })
  it('is disabled when disabled prop is true', () => { })
})

// Evitar
describe('Button', () => {
  it('works', () => { })
  it('test 1', () => { })
})
```

2. Testing Behavior, No Implementation

```
// Bueno (test behavior)
it('calls onClick when clicked', () => {
  const handleClick = vi.fn()
  render(<Button onClick={handleClick}>Click</Button>)
  fireEvent.click(screen.getByText('Click'))
  expect(handleClick).toHaveBeenCalledTimes(1)
})

// Evitar (test implementation)
it('sets onClick to button onClick prop', () => {
  const handleClick = vi.fn()
  const button = new Button({ onClick: handleClick })
  expect(button.props.onClick).toBe(handleClick)
})
```

3. Usar Testing Library Queries

```
// Bueno (prioritizado por accesibilidad)
expect(screen.getByRole('button', { name:
  'Submit' })).toBeInTheDocument()
expect(screen.getByLabelText('Email')).toBeInTheDocument()

// Acceptable (cuando no hay alternativa)
expect(screen.getByText('Submit')).toBeInTheDocument()
```



```
// Evitar (frágil)
expect(container.querySelector('.submit-
    button')).toBeInTheDocument()
```

Mejores Prácticas de Git

1. Commits Atómicos

```
# Bueno: Un commit por cambio lógico
git add src/components/Button.tsx
git commit -m "feat(button): add loading state"

git add src/components/Button.test.tsx
git commit -m "test(button): add tests for loading state"

# Evitar: Commit con cambios no relacionados
git add src/components/Button.tsx src/api/user.ts
git commit -m "various changes"
```

2. Mensajes de Commit Claros

```
# Bueno
feat(auth): add OAuth2 login support
fix(button): resolve click handler race condition
docs(readme): update installation instructions

# Evitar
fix stuff
update
wip
```

3. Branches con Nombres Descriptivos

```
# Bueno
feat/user-authentication
fix/login-button-error
refactor/optimize-api-client
docs/update-readme

# Evitar
stuff
fix-1
branch-2
```

Troubleshooting

Problemas Comunes y Soluciones

1. Error: “Cannot find module”

Problema:

Error: Cannot find module '@components/Button'

Solución: - Verifica que el path alias esté configurado en tsconfig.json y vite.config.ts - Reinicia el servidor de desarrollo - Verifica que el archivo exista en la ubicación correcta

2. Error: “TypeScript compilation failed”

Problema:

error TS2322: Type 'string' is not assignable to type 'number'

Solución: - Corrige el error de tipo en el código - Si es correcto, usa aserción de tipo: value as number - Verifica que el tipo esté correctamente definido

3. Tests Fallan Después de Refactor

Problema: Los tests fallan después de cambiar la implementación pero no el comportamiento.

Solución: - Verifica que los tests estén probando comportamiento, no implementación - Actualiza los tests si el comportamiento cambió intencionalmente - Usa snapshots con cuidado (pueden ser frágiles)

4. ESLint Error en Pre-commit

Problema: El commit se rechaza por errores de ESLint.

Solución:

```
# Ver errores
pnpm lint
```

```
# Corregir automáticamente
pnpm lint --fix
```

```
# Si no se puede corregir automáticamente, corrige manualmente
```

5. Commitlint Rechaza Commit

Problema:

```
✖ input: add new feature
✖ subject may not be empty [subject-empty]
✖ type may not be empty [type-empty]
```

Solución: Usa el formato convencional:

```
# Correcto
git commit -m "feat: add new feature"
```

```
# O usa Commitizen
pnpm commit
```

6. Port Already in Use

Problema:

Error: Port 5173 is already in use

Solución:

```
# Matar proceso en el puerto
npx kill-port 5173
```

```
# O usar otro puerto
pnpm dev --port 3000
```

7. Dependency Conflicts

Problema: Conflictos de versiones entre dependencias.

Solución:

```
# Limpiar cache e instalar
rm -rf node_modules pnpm-lock.yaml
pnpm install
```

```
# O actualizar dependencias
pnpm update
```

8. Build Failed

Problema: El build de producción falla pero el desarrollo funciona.

Solución:

```
# Verificar TypeScript
pnpm build # Esto ejecuta tsc primero

# Verificar que no haya imports relativos rotos
# Verificar que todos los archivos necesarios estén incluidos
```

Recursos Adicionales

Documentación Oficial

- **React:** <https://react.dev>
- **TypeScript:** <https://www.typescriptlang.org/docs>
- **Vite:** <https://vitejs.dev>
- **Vitest:** <https://vitest.dev>
- **Playwright:** <https://playwright.dev>
- **ESLint:** <https://eslint.org/docs/latest>
- **Prettier:** <https://prettier.io/docs/en>

Tutoriales y Guías

- **React Tutorial:** <https://react.dev/learn>
- **TypeScript Handbook:** <https://www.typescriptlang.org/docs/handbook/intro.html>
- **Vite Guide:** <https://vitejs.dev/guide/>
- **Testing Library:** <https://testing-library.com/docs/react-testing-library/intro/>

Comunidades

- **React Discord:** <https://react.dev/community>
- **TypeScript Community:** <https://www.typescriptlang.org/community>
- **Vite Discord:** <https://chat.vitejs.dev>
- **Stack Overflow:** Etiquetas: react, typescript, vite

Herramientas Útiles

- **TypeScript VSCode Plugin:** Mejora el soporte de TypeScript en VSCode
 - **ESLint VSCode Plugin:** Integración de ESLint en VSCode
 - **Prettier VSCode Plugin:** Formateo automático en VSCode
 - **Vitest VSCode Plugin:** Integración de Vitest en VSCode
 - **React DevTools:** Herramientas de desarrollo para React
-

Conclusión

Este proyecto proporciona una base sólida y moderna para desarrollar aplicaciones React con las mejores prácticas actuales. La configuración está optimizada para:

- **Desarrollo Rápido:** HMR, build instantáneo, herramientas modernas
- **Calidad de Código:** TypeScript estricto, linting automático, testing
- **Colaboración:** Git hooks, commits convencionales, code review
- **Mantenibilidad:** Estructura clara, documentación completa, estándares

Sigue las guías y convenciones descritas en esta documentación para mantener la calidad y consistencia del código a medida que el proyecto crece.

¡Feliz desarrollo!